CS112: Wednesday 14 October, 12:15 to 1:20

DRAFT, RECONSTRUCTED FROM W06-LISTS AND ERIC'S ACCOUNT OF THE CLASS

Monday was the linked list on the slides. A Node holds one item and the address of the next one, and a List keeps the first and the last. Inserting at the front is the same few pointer moves however long the list is.

Today the List gets written, in `w06-lists`. The class starts as a header and a `.cpp` of empty methods, and by the end of the hour it can be built, grown from either end, read at both ends and destroyed, with a test for each.

PART
1. Node lives inside List
2. Constructors, and one `delete` for the whole chain
3. The getters, and the first tests
4. `append`
5. `prepend`
6. The copy constructor
7. Close

Part 1. Node lives inside List

```
typedef int Item;

class List{
    ...
    private:
        class Node {
            public:
            Node(Item it, Node *next);
            ~Node();
            Item myItem;   // the value stored in this node
            Node *myNext;  // the node after this one (nullptr if
last)
        };

        Node *myFirst;
        Node *myLast;
        unsigned mySize;
};
```

> **Talk to your neighbor.** Node is in List's `private` section, but its members are `public`. Who can see `myItem`?

Only List. Nobody outside can name a Node at all, so `public` inside it means visible to the one class that can reach it. And List needs that: a nested class gets no special access to List, and List gets none to Node's private members either. With `myItem` private, `List::append` could not touch it. Outside the class, every Node method is spelled with both names: `List::Node::Node`.

Part 2. Constructors, and one `delete` for the whole chain

An empty list has no nodes, so the default constructor sets both pointers to `nullptr` and the size to 0. Then the destructor, which is one line.

```
List::~List() {
    delete myFirst;
}
```

> **Talk to your neighbor.** The list has five nodes. `delete myFirst` frees one of them. What happens to the other four?

They go too, because of Node's destructor.

```
List::Node::~Node() {
    delete myNext;
}
```

Deleting the first node runs its destructor, which deletes the second, which deletes the third, down to the last, whose `myNext` is `nullptr`. Deleting `nullptr` does nothing, and the chain stops there.

Part 3. The getters, and the first tests

`getSize` and `isEmpty` read `mySize`. `getFirst` and `getLast` read through a pointer, which an empty list does not have, so on an empty list they throw `underflow_error`. The first test case, `"default constructor"`, checks a size of 0, `isEmpty()`, and both throws.

`make test` runs the tests. `make sanitize` runs them again under AddressSanitizer, which reports a leaked or doubly freed node.

Part 4. `append`

> **Talk to your neighbor.** A new node goes on the end. Which pointers change? Is it the same when the list is empty?

Not the same. A non-empty list hooks the new node onto the old last one. An empty list has no last node, and the new node becomes the first as well.

```
void List::append(Item it) {
    Node* newNode = new Node(it, nullptr);
    if (mySize == 0) {
        myFirst = newNode;
    } else {
        myLast->myNext = newNode;
    }
    myLast = newNode;
    mySize++;
}
```

Two sections in the test: one item, where first and last are the same value, and three items, where they are not.

Part 5. `prepend`

> **Talk to your neighbor.** The new node goes at the front. What does its `myNext` point to?

The old first node, so the constructor can take it directly.

```
void List::prepend(Item it) {
    Node* newNode = new Node(it, myFirst);
    myFirst = newNode;

    if (mySize == 0) {
        myLast = newNode;
    }
    mySize++;
}
```

The empty case is the one that touches `myLast`. The test mixes the two: `append(2)`, `prepend(1)`, `append(3)`, `prepend(0)`, and the list reads 0, 1, 2, 3.

Part 6. The copy constructor

> **Talk to your neighbor.** `List l2(l);` should give `l2` its own nodes, with the same values in the same order. What already written method does most of the work?

`append`. Walk the original from `myFirst` to `nullptr`, appending each item.

```
for (Node* ptr = orig.myFirst; ptr != nullptr; ptr = ptr->myNext)
{
    this->append(ptr->myItem);
}
```

Part 7. Close

> **The List can be built, grown from either end, read at both ends and destroyed.**
>
> **Friday 23 October:** reaching into the middle, and taking nodes out.